Milestone 6 – Concurrency and Deadlock Management

Theme 3: Distributed Telecom Resource Management System (TeleRM) · ICS 2403 Distributed Computing & Applications · Week 6 · System version S6 (evolved from S5)

0. The short version

S5 made a multi-site service chain **atomic**. It handled competition for resources in the simplest possible way: if a site does not have the CPU a request needs, it says NO at once and the whole chain aborts (*no-wait* locking). That is safe, and it cannot deadlock, but it throws work away: in our workload 94 to 98 % of all attempts ended in an abort.

Milestone 6 asks what happens when requests are allowed to **wait** for resources, as real systems do. Waiting brings the classic concurrency problems into TeleRM:

- **Race conditions.** Two requests check the same free capacity at the same moment, both see enough, both take it. The site ends up promising more than it owns.
- **Mutual exclusion.** The fix, and what it costs.
- **Deadlock.** Chain T1 holds CPU on site A and waits for site B. Chain T2 holds CPU on site B and waits for site A. Neither can ever continue, and both keep their resources locked.
- **Starvation.** A large request (a 1.5-core transcoder) waits forever because small requests keep taking the capacity as it comes free.
- **Resource contention.** Locked resources that do no useful work.

In hospital terms: two patients each have a bed in a different ward and each waits for a bed in the other's ward. The beds are blocked and nobody moves. The ways out: **detect** the standoff and send one patient home (detection and recovery), **forbid** the standoff by a rule (prevention: always ask for wards in alphabetical order, or "the older patient waits, the younger leaves"), or **just wait for a timeout**.

What we built:

- A **lock manager** at every site: a wait queue with three disciplines (greedy, FIFO, aging), and three ways of updating the ledger (atomic, locked, deliberately unsafe).
- A **resource-allocation graph** P → R (request) and R → P (assignment) built from the real running system, and a **deadlock detector** on the coordinator that runs the multi-instance detection algorithm on it and aborts a victim.
- Two **prevention** strategies: **wait-die** (timestamps) and **ordered acquisition** (a fixed order over the sites), plus the S5 **no-wait** and a plain **timeout** as baselines.
- Experiments on all of it, with the resource-utilisation analysis the brief asks for.

What we found (details in section 6):

Question	Answer
How bad is a race?	With an unsafe check-then-update, 320 simultaneous requests were **all granted** on a site with room for 40: **700 % over-commit**. A lock or an atomic step grants exactly 40.
What does mutual exclusion cost?	A lock held around a 1 ms log write limits a site to **about 820 requests/s**. Doing check and update in one step with no wait between them gives **16 700 to 45 000 requests/s**. Rule: never hold a lock across I/O.
A real deadlock, five ways	Detection broke it in **0.21 s** and committed one chain. A timeout alone left **both chains stuck for 3.17 s**, wasting 8 to 24 times more CPU·s (2 274 against 94 to 291). Wait-die and ordered acquisition prevented it.
Do deadlocks happen by themselves?	In about 50 random-workload runs the detector found **none**. The lock timeouts we saw under heavy load were **overload, not deadlock**, and a timeout

	alone cannot tell the difference. The detector can.
Starvation	With greedy queuing a large request waited up to **6 s and mostly timed out**. FIFO served it in **0.6 s** but cut small-request throughput by 46 %. Aging served it in **1.2 s** and cost 25 %.
What does waiting cost in utilisation?	With waiting, **29 to 40 %** of a site's capacity sat locked by chains that were still waiting at another site. Ordered acquisition cut that waste to **13 to 26 %**.

1. What changed from S5 to S6

Part	S5 (Week 5)	S6 (Week 6)
Request that does not fit	Vote NO immediately (no-wait)	Per request: `nowait`, `timeout`, `detect`, `waitdie` or `ordered`
Site	Ledger with an atomic check-and-lock	**Lock manager**: wait queue (greedy, fifo, aging), ledger mode (atomic, locked, unsafe)
Coordinator	Plans, votes, decides	Also: **deadlock detector** (LOCKSTATE from every site, resource-allocation graph, victim choice), ordered acquisition
Messages	PREPARE, COMMIT, ABORT …	New: **LOCKSTATE** (a site's holders and waiters); PREPARE carries `cc`, a timestamp and a wait limit
Isolation	Competing request gets NO	Competing request **waits**; a victim or a timeout ends the wait
Test hooks	Crash points	Adds a per-part **delay** to force an interleaving, and `force` (skip the coordinator's view so the sites arbitrate)

New files: `telerm/deadlock.py` (graph and detection), `m6_run.py` (experiments), `analyze_m6.py` (tables). Changed: `telerm/txn.py` (lock manager in `Participant`; detector and ordered acquisition in `Coordinator`), `telerm/manager.py` (detector flags), `telerm/txnsite.py` (race/discipline flags), `m5_cluster.py` (cluster options).

2. Concurrency model

2.1 Processes and resources

- **Processes** P_i are the transactions in flight (service chains being installed). Many run at the same time: concurrent clients on the coordinator, and for each site many concurrent PREPARE handlers in its event loop.
- **Resources** R_j are one type per site and dimension: `site0/cpu`, `site0/mem`, `site0/bw`. A type has many identical units (millicores), so TeleRM is the **multi-instance** case.
- **Request edge** P_i → R_j: the transaction waits for units of R_j. **Assignment edge** R_j → P_i: it holds units.

A real snapshot from the running system (experiment X2, taken 0.6 s after two chains were submitted; `results/M6/x2_snapshot_timeout.json`). Each chain needs 600 mc on both sites, each site has 1 000 mc:

```
Resource-allocation graph (cpu)                free units
  site0/cpu --(600)--> T00001   (assignment)    site0: 400
  site1/cpu --(600)--> T00002   (assignment)    site1: 400
  T00001 --(600)--> site1/cpu   (request)       400 free < 600 requested: must wait
  T00002 --(600)--> site0/cpu   (request)       400 free < 600 requested: must wait

Wait-for graph:  T00001 -> T00002 -> T00001     (a cycle)
Detection (matrix algorithm): deadlocked = [T00001, T00002]
```

2.2 The four conditions, and where each strategy breaks one

Coffman condition	In TeleRM	Strategy that breaks it
Mutual exclusion	A millicore is held by one transaction	(kept; this is what a ledger is)
Hold and wait	A chain keeps the CPU it got on site A while waiting on site B	`nowait` (never waits)
No preemption	A PREPARED lock is a promise and is not taken back	`detect` (aborts a victim: recovery)
Circular wait	T1 waits for T2 and T2 for T1	`ordered` (sites taken in a fixed order), `waitdie` (waits only run old → young)

2.3 Strategies

cc	Rule	Class
`nowait`	A request that does not fit is refused at once (the S5 behaviour)	avoidance of waiting
`timeout`	Wait up to `lock_wait_s`, then vote NO	recovery by timeout
`detect`	Wait. Every `detect_ms` the coordinator collects LOCKSTATE, runs the detection, and if a deadlock is seen in two consecutive rounds aborts the **youngest** transaction of the deadlocked set	detection and recovery
`waitdie`	A request waits only if its timestamp is smaller (older) than that of every PREPARED holder at the site; otherwise it dies (NO). Timestamps are the transaction numbers, which the Raft log orders globally	prevention
`ordered`	The coordinator sends PREPARE to the sites **one at a time in fixed order**; each may wait. Nobody holds a higher site while waiting for a lower one, so there is no cycle	prevention

Why wait-die cannot deadlock. A transaction waits only for younger ones. A cycle T1 → T2 → … → T1 would need timestamps that grow all the way round to the start, which is impossible.

2.4 Detection algorithm

Because a resource type has several units, a cycle in the graph is necessary but **not sufficient** for deadlock (a unit may come free somewhere else). The detector uses the matrix algorithm (Silberschatz):

```
Work   = Available (free units of every resource type)
Finish_i = (Allocation_i == 0)
repeat: pick i with Finish_i = false and Request_i <= Work
        Work = Work + Allocation_i ; Finish_i = true
deadlocked = { i : Finish_i = false }
```

A transaction that holds resources and waits for nothing (for example one that is already COMMITTED: it will end by itself) has `Request = 0`, so it always finishes and returns its units to `Work`. An overloaded but healthy system is therefore **not** reported as deadlocked. Cost: $O(n^2 \cdot r)$ for n transactions and r resource types (measured in X5).

Snapshot problem. LOCKSTATE answers from different sites are taken at slightly different moments, so the assembled graph may describe a state that never existed (a *phantom deadlock*). The detector therefore acts only on a transaction that is deadlocked in **two consecutive rounds** (`stable_rounds = 2`). The cost is one extra detection interval.

Victim policy. `youngest` (largest timestamp, least work done, and also the one wait-die would have killed) is

used throughout. `cheapest` (fewest units held) is implemented but not evaluated.

Recovery. The victim is aborted with a replicated `TXN_DECIDE ABORT` (S5), then an ABORT goes to each site, which removes the queued request. The site answers the pending PREPARE with NO (`cancelled`), the coordinator reports `deadlock_victim`, and the other chain proceeds.

2.5 Queue disciplines (starvation)

Discipline	Rule when resources are released	Effect
`greedy`	Wake any waiter that fits, in arrival order	Best utilisation. A large request can wait forever
`fifo`	Wake only the head of the queue; nobody may overtake	No starvation. Small requests wait behind a large one: lower utilisation
`aging`	Greedy, but a waiter that has waited longer than `aging_s` (1 s here) blocks overtaking	Bounded wait, and most of the utilisation of greedy

2.6 Race conditions and mutual exclusion on the ledger

The ledger update is *check free capacity, then take it*. Three versions:

`race`	Implementation	Meaning
`unsafe`	check, then `await` (1 ms: stands for the WAL write between check and update), then update without re-checking	The textbook race. Deliberately wrong
`locked`	one `asyncio.Lock` around check, the same 1 ms wait, and update	Mutual exclusion by a lock
`atomic`	check and update with no `await` between them (the S5 code)	In a single-threaded event loop this is atomic without a lock

3. Fault, failure and correctness conditions

The S5 crash tests still apply (the participant and coordinator code paths are shared). New in S6: a transaction can now sit in a **WAITING** state at a site. It holds nothing yet, is never written to the WAL, and disappears when the site restarts; the coordinator's own timeout then turns it into a NO vote. An ABORT that arrives while a request is queued removes it from the queue (state ABORTED), and a grant that races with the abort is given back.

Invariants (all must be 0 after every run): the six of S5 (atomicity, coordinator/site agreement, no over-capacity, ledger equals contents, no stuck locks, no protocol violation) plus the **race counter** (`over_capacity_events`, X1).

4. Implementation notes

- **Lock manager** (`Participant._acquire`, `_grant`, `_cancel_waiter`, `on_lockstate`): the queue is a list of waiters with a future each; every release (`_abort`, `exit_instance`, undo, cancel) calls `_grant()`, which wakes waiters according to the discipline.
- **Detector** (`Coordinator.detector_loop`): runs on the Raft leader only. Every round costs `2m` messages for `m` sites.
- **Prevention hooks**: `waitdie` is decided at the site (it knows its holders); `ordered` is decided at the coordinator (it sequences the PREPAREs).
- **`force`**: in S5 the coordinator refuses a request at planning when its (optimistic) view says "full". For S6 experiments a part may set `force`, which skips that check so that the *sites* decide. Without it the admission control hides the concurrency problem.
- **Test hook** `debug_delay_ms` on a part delays that site's handling of PREPARE, so two chains can be made to interleave in the order that produces a deadlock.

5. Experimental method

Same cluster as Milestone 5 (separate OS processes, **one CPU core**, emulated delays; `m5_cluster.py`). Every experiment: hypothesis, design, implementation, measurement, analysis, conclusion.

6. Experiments

X1 Race condition and mutual exclusion

Hypothesis. An unsafe ledger over-commits as soon as more requests arrive together than there is room for; a lock or an atomic step never does; a lock around I/O is much slower than an atomic step. **Design.** One site with 4 000 mc (room for 40 requests of 100 mc). `c` requests arrive at the same instant (10 to 320), in-process so that scheduling noise is minimal. 5 repetitions per cell.

Concurrent requests	unsafe: granted / over-commit	locked: granted / requests per s	atomic: granted / requests per s
10	10 / 0 %	10 / 814	10 / 16 685
40	40 / 0 %	40 / 824	40 / 31 017
80	**80 / 100 %**	40 / 1 613	40 / 42 265
160	**160 / 300 %**	40 / 3 132	40 / 45 196
320	**320 / 700 %**	40 / 5 737	40 / 43 847

Analysis. The unsafe version grants everything: every request passes the check before the first one updates the ledger (the `await` lets them all in). The site then “owns” 32 000 mc on a 4 000 mc machine, and the stride scheduler (S1) would try to give shares of a CPU that does not exist. Both safe versions grant exactly 40. The lock serialises the 1 ms critical section (about 820 requests/s at low load; the higher numbers at high load are the rejected requests, which leave without taking the lock). The atomic version has the same correctness at 20 to 40 times the rate (10 to 40 requests), because the check and update take microseconds and nothing is held across the I/O. **Conclusion.** Hypothesis confirmed. **Make the check-and-update step indivisible, and keep I/O out of the critical section.** A lock is the right tool only where the critical section must contain a wait.

X2 A real distributed deadlock, five ways

Hypothesis. Without any mechanism a deadlock lasts until the timeout and wastes the most resource; detection ends it in about one detection interval; prevention avoids it; no-wait avoids it by aborting. **Design.** Two sites with 1 000 mc each. Chain T1 needs 600 mc on site0 (immediately) and on site1 (150 ms later); T2 needs site1 first and site0 150 ms later. T1 is submitted 10 ms before T2 (older). Holding 1 s, lock wait 3 s, detector every 200 ms with two stable rounds. 5 repetitions per strategy (detect: 8, because three runs at 200 ms come from X5). “Wasted” is the CPU·s held by transactions that ended in an abort.

Strategy	Outcome	Time until both clients have an answer	Wasted CPU·s	Notes
`nowait`	T1 aborted (node_capacity), T2 committed	0.17 s	94	no deadlock possible; one chain lost
`timeout`	both aborted (1 of 5 runs) or one aborted by timeout then the other committed (4 of 5)	**3.17 s**	**2 274**	stuck until the timeout; both hold their CPU the whole time
`detect`	T1 committed, T2 `deadlock_victim`	0.51 s	291	1 deadlock found; victim aborted **0.207 s** after it was first seen
`waitdie`	T1 committed, T2 aborted (wait_die)	0.17 s	93	younger chain dies at once
`ordered`	**both committed**	1.18 s	**0**	T2 waits at site0 holding nothing; runs after T1’s 1 s holding time

No run violated an invariant (atomicity, capacity, stuck locks).

Analysis. Detection turns a 3.2 s standoff into a 0.2 s one and saves the other chain. Prevention by wait-die is as quick but sacrifices the younger chain without having checked whether it was really deadlocked. Ordered acquisition is the only strategy where **both** chains succeed and nothing is wasted, but T2 was made to wait for T1's whole holding time because it queued at the lower site. No-wait never waits and loses one chain at once. **Conclusion.** Hypothesis confirmed. Every strategy was safe; they differ in what they sacrifice.

X3 Strategies under a random multi-site workload; resource utilisation

Hypothesis. Waiting strategies commit more of what is offered than no-wait but lock resources that do no work while chains wait; deadlock handling matters in proportion to how often cycles form. **Design.** 4 sites, chains of 2 or 3 parts of 1 000 mc on random sites (0.3 s holding), 1 ms delay, lock wait 1.5 s, 4, 8 and 16 closed-loop clients for 10 s, `force` on. Two capacities: **X3** 2 000 mc per site (two parts fit) and **X3b** 1 000 mc per site (one part fits: every site is a lock). A sampler reads LOCKSTATE of every site 4 times a second: *locked* = capacity held by PREPARED or running instances, *useful* = capacity of running instances, *wasted* = locked − useful (held by chains that are still collecting their locks). X3: 2 repetitions. X3b: 1 repetition (a second one was cut off after 3 runs and is not used).

X3 (two slots per site), mean over 2 runs; "ratio" = committed / offered:

Clients	Strategy	commits/s	ratio	p95 ms	mean lock wait ms	locked / useful / wasted	abort reasons
4	nowait	8.3	0.06	34	-	0.84 / 0.68 / 0.17	2 630 node_capacity
4	timeout	7.4	1.00	630	447	0.98 / 0.68 / 0.29	none
4	detect	7.4	1.00	630	446	0.97 / 0.67 / 0.31	none
4	waitdie	7.2	0.33	598	264	0.92 / 0.63 / 0.29	297 wait_die
4	ordered	7.7	1.00	803	381	0.82 / 0.69 / **0.13**	none
16	nowait	7.6	0.05	129	-	0.88 / 0.62 / 0.26	2 791 node_capacity
16	timeout	7.1	0.63	1 509	1 177	0.98 / 0.62 / 0.36	95 lock_timeout
16	detect	7.1	0.63	1 484	1 181	0.98 / 0.62 / 0.36	94 lock_timeout
16	waitdie	6.0	0.09	881	411	0.95 / 0.52 / 0.43	1 293 wait_die
16	ordered	7.2	0.58	1 950	639	0.88 / 0.65 / **0.23**	118 lock_timeout

X3b (one slot per site), 1 run per cell:

Clients	Strategy	commits/s	ratio	p95 ms	locked / useful / wasted	abort reasons
4	nowait	3.9	0.03	34	0.83 / 0.61 / 0.22	2 643 node_capacity
4	timeout / detect	3.5 / 3.6	1.00	1 245	0.99 / 0.64 / 0.34	none
4	waitdie	3.5	0.33	985	0.95 / 0.57 / 0.38	75 wait_die
4	ordered	3.5	0.91	1 563	0.76 / 0.63 / **0.13**	4 lock_timeout
16	nowait	3.5	0.02	143	0.89 / 0.55 / 0.34	1 490 node_capacity

16	timeout / detect	3.4 / 3.5	0.33	1 526	0.99 / 0.61 / 0.38	81 lock_timeout
16	waitdie	3.2	0.10	1 304	0.96 / 0.51 / 0.45	290 wait_die, 28 lock_timeout
16	ordered	3.5	0.32	2 021	0.88 / 0.57 / **0.31**	84 lock_timeout

All runs: 0 invariant violations. **The detector reported 0 deadlocks in every run.**

Analysis.

- **Throughput is set by capacity, not by the strategy.** Across all strategies and loads it ranges from 6.0 to 8.3 commits/s with two slots and from 3.0 to 3.9 with one; no strategy is a clear winner. The sites are full whatever we do.
- **What differs is the work thrown away.** No-wait commits 2 to 6 % of its attempts: 2 630 aborted attempts for about 83 commits, each costing a coordinator step, a PREPARE round and an ABORT round. Waiting strategies convert those aborts into waiting.
- **Waiting has a price in latency and in locked-but-idle capacity.** p95 latency rises from 34 to 143 ms (no-wait) to 600 to 2 000 ms. With `timeout` and `detect`, 29 to 40 % of every site's capacity was locked by chains that were still waiting for another site (*wasted* column). Ordered acquisition cut it to 13 to 31 %, because a chain holds only the sites it has already finished collecting and cannot sit on a high site while queuing for a low one.
- **Wait-die kills more than it needs to.** It aborts 67 to 91 % of attempts here, against 0 to 67 % for timeout/detect/ordered, because it dies on *possible* deadlock, not actual deadlock. Its latency is the lowest of the waiting strategies, and its wasted fraction at 16 clients is the worst (0.43 to 0.45).
- **No deadlock formed.** Every PREPARE of a chain goes out in parallel and arrives within about a millisecond; a cycle needs the specific interleaving that X2 forced with a 150 ms delay. In this workload the lock timeouts at 16 clients (81 to 95 per run) are **congestion**: `detect` reported nothing, and had the same abort count as `timeout`. A timeout cannot tell the two apart. The detector can, and it did not fire.

Conclusion. Hypothesis partly confirmed. Waiting beats no-wait on wasted attempts; ordered acquisition wins on wasted capacity; on this workload deadlock itself is rare and the deadlock handling matters when the interleaving is unlucky (X2), not on average.

X4 Starvation

Hypothesis. Greedy queuing starves a large request among small ones; FIFO and aging bound its wait, FIFO at a higher cost in utilisation. **Design.** One site, 2 000 mc. Eight closed-loop clients each request 250 mc (0.3 s) back to back; one client requests 1 500 mc (0.5 s) repeatedly, with a 6 s wait limit. 16 s per run, 2 repetitions per discipline, aging at 1 s.

Discipline	Large requests: attempts / committed / timed out	Large wait, mean / max	Small commits/s	Small p95 ms	Capacity locked / running
greedy	3.0 / 1.0 / **2.0**	3.11 s / **6.01 s**	**25.8**	308	0.993 / 0.976
fifo	18 / 18 / 0	0.61 s / 0.63 s	13.9 (−46 %)	817	0.939 / 0.915
aging (1 s)	10 / 10 / 0	1.23 s / 1.25 s	19.4 (−25 %)	813	0.963 / 0.950

Analysis. Under greedy, small requests refill the freed capacity at once; the 1 500 mc request needs 1 500 free mc and rarely sees it. It almost never saw 1 500 free mc, and timed out twice. FIFO treats arrival order strictly: the large request is served within 0.6 s, but while the queue drains for it the site sits partly idle (capacity locked drops from 99 % to 94 %) and small requests lose 46 % of their throughput. Aging allows overtaking for the first second and then reserves the head: a bounded wait (about the aging time plus service time), at a cost of 25 % of small throughput. **Conclusion.** Hypothesis confirmed. Starvation is a policy choice with a measurable price; aging is the compromise.

X5 Cost and behaviour of the detector

(a) Algorithm cost (in-process, 16 sites, best of 3):

chain (worst case: one finishes

Transactions	ring (all deadlocked)	per pass)
10	0.009 ms	0.018 ms
100	0.08 ms	1.2 ms
1 000	0.9 ms	136 ms
3 000	3.7 ms	1 420 ms

The ring case is a single pass (linear). The chain case needs one pass per transaction and grows quadratically, as the analysis predicts. TeleRM deals with tens of waiting chains, where detection takes microseconds.

(b) Detection interval against time to resolve (X2 scenario):

Interval	Time from first sight of the deadlock to the victim’s abort	Time until both clients have an answer	Detector messages per second (2 sites)
50 ms	0.056 s	0.27 s	80
100 ms	0.106 s	0.32 s	40
200 ms	0.207 s	0.51 s	20
500 ms	0.507 s	0.87 s	8
1 000 ms	1.006 s	2.07 s	4

Resolution takes about one interval after the deadlock is first seen (the stability rule waits for the second round). Messages per second are `2m / interval` for `m` sites. Shortening the interval buys faster resolution for linear message cost.

(c) False positives. A workload that cannot deadlock (ordered acquisition, 16 clients, detector every 100 ms), 3 runs of 10 s each, with `stable_rounds` 1 and 2: 293 transactions, 365 and 371 detection rounds, **0 deadlocks reported and 0 victims** in both. We could not provoke a phantom deadlock, so the benefit of the two-round rule is **not demonstrated** by our data; it stays as a precaution justified by the theory.

7. Baseline against proposed system

	Baseline: S5 no-wait locking	Proposed: waiting with a lock manager, detection, aging, ordered acquisition
Attempts that end in abort (X3, 4 clients)	94 to 97 %	0 % (`detect`, `timeout`, `ordered`)
Throughput (X3, 4 clients)	8.3 /s	7.2 to 7.7 /s (7 to 13 % lower, capacity bound)
p95 latency (X3, 4 clients)	34 ms	600 to 800 ms
Wasted capacity (X3, 16 clients)	0.26	0.36 (`detect`), **0.23** (`ordered`)
Large request among small ones (X4)	served only with no-wait (refused when full)	served in 0.6 s (fifo) or 1.2 s (aging)
Deadlock resolution (X2)	never forms	0.21 s with detection, 3.17 s with timeout only
Safety (all runs)	0 violations	0 violations

The honest reading: waiting does not raise throughput on a full system, it **removes the wasted aborts and the starvation** at the price of latency and of locked idle capacity, and ordered acquisition is the strategy that gives most of the benefit with the least waste. The new engineering elements are the multi-instance detection on a global resource-allocation graph, the two-round stability rule, and the aging queue, each measured.

8. Failure-driven engineering log – Week 6

	Entry 1	Entry 2	Entry 3	Entry 4
What changed?	First workload with waiting	Race experiment	Queue discipline	Workload with more contention
		The lock version was	A smoke run with	Still no deadlock in

What failed?	No request ever waited: all strategies behaved like no-wait	20 to 40 times slower than the atomic one	greedy queuing: the large request timed out	about 50 runs, although timeouts appeared
Why?	The coordinator's planner already refused requests its view said would not fit, so nothing reached the sites' queues	The lock was held across the 1 ms wait that stands for the WAL write	A 1 500 mc request needs 1 500 free; small requests refilled the capacity first	Chain PREPAREs go out in parallel and arrive about 1 ms apart; a cycle needs a particular interleaving
How fixed?	Added `force`: skip the view check so the sites arbitrate	Not a bug: reported; the atomic variant keeps the check-and-update step free of I/O	Added FIFO and aging disciplines	Built the staggered two-chain scenario (X2) to produce a real deadlock; reported the workload result as it is
Alternative considered	Count PREPARED locks in the planner's view	Lock only the in-memory step, do the I/O outside	Priority by size	A distributed edge-chasing detector (Chandy–Misra–Haas)
What was learned?	Admission control at a central point hides concurrency problems at the sites	Correctness comes from indivisibility, not from locks; locks around I/O are a performance bug	Starvation is a scheduling-policy property that must be measured	A timeout cannot tell deadlock from overload; a detector can

9. Limitations of S6

- **Centralised detector.** It runs on the Raft leader and needs a LOCKSTATE answer from every site; if a site does not answer, its part of the graph is missing, and the detection round simply finds less. An edge-chasing algorithm would avoid collecting the whole graph (alternative considered, not built).
- **Phantom deadlocks not provoked.** The two-round rule is in place but our data cannot show it matters.
- **Only the youngest-victim policy was evaluated.** `cheapest` exists in the code. **Wound-wait** (preempting a holder) is not implemented: a PREPARED lock is a promise, and preempting it needs an abort protocol.
- **Natural deadlocks were not observed.** The strategy comparison (X3) is mostly a comparison of how waiting behaves under overload; the deadlock behaviour is shown by the forced scenario (X2).
- **X3b has one repetition**, and the second was cut short.
- **Waiting is per site only.** The lock manager does not model lock modes (shared/exclusive); every request is an exclusive claim on a quantity.
- **Not run on the real sites.** The lock manager is inside the shared `Participant` class used by `site.py`, but the S6 experiments use the light participant process, as in Milestone 5.
- **One machine, one core**, emulated delays. Ratios are meaningful; absolute rates are specific to the machine.

10. Reproducibility record

Item	Value
Hardware / OS	1 vCPU, Ubuntu 24 (Linux sandbox)
Language / libraries	Python 3.12, standard library only
Topology	single host, loopback; ports 8900+ (coordinators), 8910 (registry), 8920+ (sites); 1 ms links; per-site delay emulated
Configuration	per experiment in `m6_run.py`; copied into every result folder (`logs/`, `wal/`)
Workload	X1 simultaneous PREPAREs; X2 two crossing chains; X3 random chains of 1 000 mc, 0.3 s; X4 8 small + 1 large client
Random seed	42 (`--seed`)

Parameters	election 300–600 ms, heartbeat 50 ms; lock wait 3 s (X2) / 1.5 s (X3) / 6 s (X4); detector 200 ms (50–1000 in X5), `stable_rounds` 2; aging 1 s
Command	`./run_m6_all.sh` (about 35 minutes)
Output	`results/M6/*.csv`, `results/M6/summary_tables.txt` (`python3 analyze_m6.py results/M6`)

11. How to run

```
python3 m6_run.py smoke                 # one deadlock, detected and resolved
python3 m6_run.py race                  # X1
python3 m6_run.py deadlock --reps 5     # X2
python3 m6_run.py workload --reps 2     # X3 (two slots per site)
python3 m6_run.py workload1 --reps 2    # X3b (one slot per site)
python3 m6_run.py starvation --reps 2   # X4
python3 m6_run.py detector --reps 3     # X5
python3 analyze_m6.py results/M6        # tables
```

A request: `TXN_REQUEST {"request": {"parts": [...], "proto": "2pc", "cc": "detect", "lock_wait_s": 3.0}}`. The detector is switched on with `--detect-ms 200` on the orchestrators; queue discipline and ledger mode with `--discipline` and `--race` on the participants.